Ejercicios POO Avanzada — Unidad 4

En esta guía encontrarás una serie de prácticas para ser realizadas de la mano de la Unidad 4 de Programación III. Los ejercicios se organizan siguiendo el orden temático de las clases: herencia, colecciones, sobreescritura, interfaces, asociación y modelo de dominio. Repasá el contenido cuando sea necesario y cualquier consulta, no dudes en preguntar.

1) Jerarquía de Animales (Herencia)

Clase base: Animal **Clases derivadas:** Perro, Gato

Propiedades de Animal:

- string Nombre
- int Edad
- string Especie

Constructores de Animal:

- Vacío: valores por defecto.
- Con Nombre, Edad y Especie (validar Edad >= 0).

Método:

- void Describir(): imprime Nombre, Edad y Especie.

Clases derivadas:

- **Perro:** propiedad adicional string Raza. Constructor con Nombre, Edad y Raza.
- **Gato:** propiedad adicional bool EsDeInterior. Constructor con Nombre, Edad y EsDeInterior.

Pruebas mínimas:

1. Instanciar un Perro y un Gato con el constructor parametrizado.
2. Llamar Describir() en cada uno desde Main.
3. Verificar que Describir() en la clase derivada también muestra la propiedad propia (Raza / EsDeInterior).

2) Listado de Animales (Colecciones)

Partiendo de las clases del ejercicio anterior, trabajar con colecciones genéricas.

Tareas:

- Crear una List<Animal> con al menos 3 Perros y 2 Gatos con distintas edades.
- Agregar un Animal más a la lista después de crearla.
- Eliminar de la lista todos los animales cuya Edad sea menor a 2 años.
- Recorrer la lista con foreach y llamar Describir() en cada elemento.
- Usar LINQ para filtrar y mostrar solo los animales con Edad > 3.

Pruebas mínimas:

4. Mostrar el conteo de la lista antes y después de la eliminación.
5. Verificar que el filtro LINQ no modifica la lista original.

3) Sonidos y Polimorfismo (Sobreescritura de métodos)

Extender las clases del ejercicio 1 para demostrar sobreescritura con override y polimorfismo en tiempo de ejecución.

Modificaciones:

- En Animal: marcar Describir() como virtual.
- En Perro: sobreescribir Describir() con override. Llamar a base.Describir() primero y luego agregar "Raza: {Raza}".
- En Gato: ídem, agregar "Interior: {EsDeInterior}".

Pruebas mínimas:

6. Crear una List<Animal> con instancias mezcladas de Perro y Gato.
7. Recorrer la lista y llamar Describir() en cada elemento.
8. Observar que cada objeto ejecuta su propia versión del método (polimorfismo).

4) Atención Veterinaria (Interfaces)

Interfaz: IAtendible

Método de la interfaz:

- void Atender(): a implementar en cada clase.

Implementaciones:

- **Perro:** implementa IAtendible. Atender() imprime "Atendiendo al perro {Nombre}: revisión general y vacunas".
- **Gato:** implementa IAtendible. Atender() imprime "Atendiendo al gato {Nombre}: desparasitación".
- **Conejo (nueva clase):** implementa IAtendible. Propiedades Nombre y Peso (decimal). Atender() imprime "Atendiendo al conejo {Nombre}: control de peso".

Pruebas mínimas:

9. Crear una List<IAtendible> con instancias de Perro, Gato y Conejo.
10. Recorrer la lista y llamar Atender() en cada elemento.
11. Reflexionar: ¿por qué Conejo puede estar en esa lista aunque no herede de Animal?

5) Turno Veterinario (Asociación)

Modelar un turno de atención usando asociación entre clases (no herencia).

Clase Veterinario:

- string Nombre
- string Matricula
- Constructor con ambas propiedades.

Clase Turno:

- Animal Paciente (asociación)
- Veterinario Profesional (asociación)
- DateTime FechaHora
- Constructor con los tres valores.
- void MostrarResumen(): imprime todos los datos del turno, incluyendo nombre del animal y nombre del veterinario.

Pruebas mínimas:

12. Instanciar un Veterinario, un Perro y un Turno que los relacione.
13. Llamar MostrarResumen() y verificar que muestra datos de los tres objetos.
14. Crear un segundo turno para el mismo Veterinario con un Gato diferente.

6) Modelo de Dominio (Diagrama)

Sin escribir código, representar gráficamente el modelo de dominio construido a lo largo de los ejercicios anteriores. Podés usar papel, draw.io o cualquier herramienta de diagramas.

Clases a incluir:

- Animal, Perro, Gato, Conejo
- Veterinario, Turno
- IAtendible

Relaciones a rotular:

- **Herencia (flecha con triángulo):** Perro → Animal, Gato → Animal.
- **Implementación de interfaz (línea punteada):** Perro, Gato, Conejo → IAtendible.
- **Asociación (línea simple con multiplicidad):** Turno → Animal (1..1), Turno → Veterinario (1..1).

Entrega:

- Foto del papel o exportación en imagen/PDF del diagrama digital.
- Estar listo para explicar en clase por qué elegiste cada tipo de relación.

7) Refactoring (Tipos de Clases)

Aplicar tipos de clases especiales para mejorar el diseño del sistema.

Modificaciones:

- Convertir Animal en clase abstract (no se instancia directamente).
- Agregar método abstracto string HacerSonido() en Animal.
- Implementar HacerSonido() en Perro ("Guau"), Gato ("Miau") y Conejo ("Squeak").
- Crear clase static RegistroVeterinaria con método estático int GenerarId() que retorne un número incremental por cada turno creado.

Pruebas mínimas:

15. Verificar que intentar instanciar Animal directamente genera error de compilación.
16. Llamar HacerSonido() sobre una List<Animal> mixta y observar el resultado.
17. Crear 3 turnos y verificar que GenerarId() retorna 1, 2 y 3 respectivamente.

8) Cliente y Reflexión de Diseño

Ejercicio de cierre: ampliar el modelo y reflexionar sobre decisiones de diseño pensando en la próxima unidad (capas y SQL).

Clase Cliente:

- string Nombre
- string Telefono
- List<Animal> Mascotas (un cliente puede tener varios animales).
- void AgregarMascota(Animal a): agrega a la lista.
- void MostrarMascotas(): lista el nombre y especie de cada mascota.

Preguntas de reflexión (para discutir en clase):

- ¿Qué otras relaciones faltan en el modelo para que sea realista?
- ¿Cómo representarías la relación Cliente-Animal si pasás el modelo a tablas SQL?
- ¿Tiene sentido que Turno tenga al Cliente o alcanza con tener al Animal?
- ¿Qué clases agregarías en una Unidad 5 con WinForms y base de datos?

Nota: no hay código obligatorio en este ejercicio. Se espera reflexión escrita o participación oral en clase.